

Getting started

This chapter is here to help you get started with Polars. It covers all the fundamental features and functionalities of the library, making it easy for new users to familiarise themselves with the basics from initial installation and setup to core functionalities. If you're already an advanced user or familiar with dataframes, feel free to skip ahead to the [next chapter about installation options](#).

Installing Polars

Python

```
pip install polars
```

Rust

```
cargo add polars -F lazy

# Or Cargo.toml
[dependencies]
polars = { version = "x", features = ["lazy", ...]}
```

Reading & writing

Polars supports reading and writing for common file formats (e.g., csv, json, parquet), cloud storage (S3, Azure Blob, BigQuery) and databases (e.g., postgres, mysql). Below, we create a small dataframe and show how to write it to disk and read it back.

Python

[API](#) [DataFrame](#)

```
import polars as pl
import datetime as dt

df = pl.DataFrame(
    {
```

```

    "name": ["Alice Archer", "Ben Brown", "Chloe Cooper", "Daniel Donovan"],
    "birthdate": [
        dt.date(1997, 1, 10),
        dt.date(1985, 2, 15),
        dt.date(1983, 3, 22),
        dt.date(1981, 4, 30),
    ],
    "weight": [57.9, 72.5, 53.6, 83.1], # (kg)
    "height": [1.56, 1.77, 1.65, 1.75], # (m)
}
)

print(df)

```



RPI DataFrame

```

use chrono::prelude::*;
use polars::prelude::*;

let mut df: DataFrame = df!(
    "name" => ["Alice Archer", "Ben Brown", "Chloe Cooper", "Daniel Donovan"],
    "birthdate" => [
        NaiveDate::from_ymd_opt(1997, 1, 10).unwrap(),
        NaiveDate::from_ymd_opt(1985, 2, 15).unwrap(),
        NaiveDate::from_ymd_opt(1983, 3, 22).unwrap(),
        NaiveDate::from_ymd_opt(1981, 4, 30).unwrap(),
    ],
    "weight" => [57.9, 72.5, 53.6, 83.1], // (kg)
    "height" => [1.56, 1.77, 1.65, 1.75], // (m)
)
.unwrap();
println!("{df}");

```

shape: (4, 4)

name	birthdate	weight	height
---	---	---	---
str	date	f64	f64
Alice Archer	1997-01-10	57.9	1.56
Ben Brown	1985-02-15	72.5	1.77
Chloe Cooper	1983-03-22	53.6	1.65
Daniel Donovan	1981-04-30	83.1	1.75

In the example below we write the dataframe to a csv file called `output.csv`. After that, we

read it back using `read_csv` and then print the result for inspection.

Python

[API read_csv](#) · [API write_csv](#)

```
df.write_csv("docs/assets/data/output.csv")
df_csv = pl.read_csv("docs/assets/data/output.csv", try_parse_dates=True)
print(df_csv)
```

Rust

[API CsvReader](#) · [API CsvWriter](#) · [Available on feature csv](#)

```
use std::fs::File;

let mut file = File::create("docs/assets/data/output.csv").expect("could not
create file");
CsvWriter::new(&mut file)
    .include_header(true)
    .with_separator(b',')
    .finish(&mut df)?;
let df_csv = CsvReadOptions::default()
    .with_has_header(true)
    .with_parse_options(CsvParseOptions::default().with_try_parse_dates(true))
    .try_into_reader_with_file_path(Some("docs/assets/data/output.csv".into()))?
    .finish()?;
println!("{df_csv}");
```

shape: (4, 4)

name	birthdate	weight	height
---	---	---	---
str	date	f64	f64
Alice Archer	1997-01-10	57.9	1.56
Ben Brown	1985-02-15	72.5	1.77
Chloe Cooper	1983-03-22	53.6	1.65
Daniel Donovan	1981-04-30	83.1	1.75

For more examples on the CSV file format and other data formats, see the [IO section](#) of the user guide.

Expressions and contexts

Expressions are one of the main strengths of Polars because they provide a modular and flexible way of expressing data transformations.

Here is an example of a Polars expression:

```
pl.col("weight") / (pl.col("height") ** 2)
```

As you might be able to guess, this expression takes the column named “weight” and divides its values by the square of the values in the column “height”, computing a person's BMI. Note that the code above expresses an abstract computation: it's only inside a Polars *context* that the expression materializes into a series with the results.

Below, we will show examples of Polars expressions inside different contexts:

- `select`
- `with_columns`
- `filter`
- `group_by`

For a more [detailed exploration of expressions and contexts](#) see the respective user guide [section](#).

`select`

The context `select` allows you to select and manipulate columns from a dataframe. In the simplest case, each expression you provide will map to a column in the result dataframe:

Python

[API select](#) · [API alias](#) · [API dt namespace](#)

```
result = df.select(  
    pl.col("name"),  
    pl.col("birthdate").dt.year().alias("birth_year"),  
    (pl.col("weight") / (pl.col("height") ** 2)).alias("bmi"),  
)  
print(result)
```

Rust

[API select](#) · [API alias](#) · [API dt namespace](#) · [Available on feature temporal](#)

```
let result = df
  .clone()
  .lazy()
  .select([
    col("name"),
    col("birthdate").dt().year().alias("birth_year"),
    (col("weight") / col("height").pow(2)).alias("bmi"),
  ])
  .collect()?;
println!("{result}");
```

shape: (4, 3)

name	birth_year	bmi
---	---	---
str	i32	f64
Alice Archer	1997	23.791913
Ben Brown	1985	23.141498
Chloe Cooper	1983	19.687787
Daniel Donovan	1981	27.134694

Polars also supports a feature called “expression expansion”, in which one expression acts as shorthand for multiple expressions. In the example below, we use expression expansion to manipulate the columns “weight” and “height” with a single expression. When using expression expansion you can use `.name.suffix` to add a suffix to the names of the original columns:

Python

[API select](#) · [API alias](#) · [API name namespace](#)

```
result = df.select(
    pl.col("name"),
    (pl.col("weight", "height") * 0.95).round(2).name.suffix("-5%"),
)
print(result)
```

Rust

[API select](#) · [API alias](#) · [API name namespace](#) · [Available on feature lazy](#)

```

let result = df
  .clone()
  .lazy()
  .select([
    col("name"),
    (cols(["weight", "height"]).as_expr() * lit(0.95))
      .round(2, RoundMode::default())
      .name()
      .suffix("-5%"),
  ])
  .collect()?;
println!("{result}");

```

shape: (4, 3)

name	weight-5%	height-5%
---	---	---
str	f64	f64
Alice Archer	55.0	1.48
Ben Brown	68.88	1.68
Chloe Cooper	50.92	1.57
Daniel Donovan	78.94	1.66

You can check other sections of the user guide to learn more about [basic operations](#) or [column selections in expression expansion](#).

with_columns

The context `with_columns` is very similar to the context `select` but `with_columns` adds columns to the dataframe instead of selecting them. Notice how the resulting dataframe contains the four columns of the original dataframe plus the two new columns introduced by the expressions inside `with_columns`:



API with_columns

```

result = df.with_columns(
    birth_year=pl.col("birthdate").dt.year(),
    bmi=pl.col("weight") / (pl.col("height") ** 2),
)
print(result)

```

 RustAPI `with_columns`

```
let result = df
    .clone()
    .lazy()
    .with_columns([
        col("birthdate").dt().year().alias("birth_year"),
        (col("weight") / col("height").pow(2)).alias("bmi"),
    ])
    .collect()?;
println!("{result}");
```

shape: (4, 6)

name	birthdate	weight	height	birth_year	bmi
---	---	---	---	---	---
str	date	f64	f64	i32	f64
Alice Archer	1997-01-10	57.9	1.56	1997	23.791913
Ben Brown	1985-02-15	72.5	1.77	1985	23.141498
Chloe Cooper	1983-03-22	53.6	1.65	1983	19.687787
Daniel Donovan	1981-04-30	83.1	1.75	1981	27.134694

In the example above we also decided to use named expressions instead of the method `alias` to specify the names of the new columns. Other contexts like `select` and `group_by` also accept named expressions.

`filter`

The context `filter` allows us to create a second dataframe with a subset of the rows of the original one:

 PythonAPI `filter` · API `dt namespace`

```
result = df.filter(pl.col("birthdate").dt.year() < 1990)
print(result)
```

[API filter](#) · [API dt namespace](#) · [Available on feature temporal](#)

```
let result = df
    .clone()
    .lazy()
    .filter(col("birthdate").dt().year().lt(lit(1990)))
    .collect()?;
println!("{result}");
```

shape: (3, 4)

name	birthdate	weight	height
---	---	---	---
str	date	f64	f64
Ben Brown	1985-02-15	72.5	1.77
Chloe Cooper	1983-03-22	53.6	1.65
Daniel Donovan	1981-04-30	83.1	1.75

You can also provide multiple predicate expressions as separate parameters, which is more convenient than putting them all together with `&`:

Python

[API filter](#) · [API is_between](#)

```
result = df.filter(
    pl.col("birthdate").is_between(dt.date(1982, 12, 31), dt.date(1996, 1, 1)),
    pl.col("height") > 1.7,
)
print(result)
```

Rust

[API filter](#) · [API is_between](#) · [Available on feature is_between](#)

```
let result = df
    .clone()
    .lazy()
    .filter(
        col("birthdate")
            .is_between(
```



```

        lit(NaiveDate::from_ymd_opt(1982, 12, 31).unwrap()),
        lit(NaiveDate::from_ymd_opt(1996, 1, 1).unwrap()),
        ClosedInterval::Both,
    )
    .and(col("height").gt(lit(1.7))),
)
.collect()?;
println!("{result}");

```

shape: (1, 4)

name	birthdate	weight	height
---	---	---	---
str	date	f64	f64
Ben Brown	1985-02-15	72.5	1.77

group_by

The context `group_by` can be used to group together the rows of the dataframe that share the same value across one or more expressions. The example below counts how many people were born in each decade:

Python

[API group_by](#) · [API alias](#) · [API dt namespace](#)

```

result = df.group_by(
    (pl.col("birthdate").dt.year() // 10 * 10).alias("decade"),
    maintain_order=True,
).len()
print(result)

```

Rust

[API group_by](#) · [API alias](#) · [API dt namespace](#) ·  Available on feature temporal

```

// Use `group_by_stable` if you want the Python behaviour of
// `maintain_order=True`.
let result = df
    .clone()
    .lazy()
    .group_by([(col("birthdate").dt().year() / lit(10) *

```

```
lit(10)).alias("decade"))))
    .agg([len()])
    .collect()?;
println!("{result}");
```

shape: (2, 2)

decade	len
---	---
i32	u32
1990	1
1980	3

The keyword argument `maintain_order` forces Polars to present the resulting groups in the same order as they appear in the original dataframe. This slows down the grouping operation but is used here to ensure reproducibility of the examples.

After using the context `group_by` we can use `agg` to compute aggregations over the resulting groups:

Python

`API` `group_by` · `API` `agg`

```
result = df.group_by(
    (pl.col("birthdate").dt.year() // 10 * 10).alias("decade"),
    maintain_order=True,
).agg(
    pl.len().alias("sample_size"),
    pl.col("weight").mean().round(2).alias("avg_weight"),
    pl.col("height").max().alias("tallest"),
)
print(result)
```

Rust

`API` `group_by` · `API` `agg`

```
let result = df
    .clone()
    .lazy()
    .group_by([(col("birthdate").dt().year() / lit(10) *
lit(10)).alias("decade")])
```

```

    .agg([
        len().alias("sample_size"),
        col("weight")
            .mean()
            .round(2, RoundMode::default())
            .alias("avg_weight"),
        col("height").max().alias("tallest"),
    ])
    .collect()?;
println!("{result}");

```

shape: (2, 4)

decade	sample_size	avg_weight	tallest
---	---	---	---
i32	u32	f64	f64
1990	1	57.9	1.56
1980	3	69.73	1.77

More complex queries

Contexts and the expressions within can be chained to create more complex queries according to your needs. In the example below we combine some of the contexts we have seen so far to create a more complex query:

Python

`API group_by` · `API agg` · `API select` · `API with_columns` · `API str namespace` · `API list namespace`

```

result = (
    df.with_columns(
        (pl.col("birthdate").dt.year() // 10 * 10).alias("decade"),
        pl.col("name").str.split(by=" ").list.first(),
    )
    .select(
        pl.all().exclude("birthdate"),
    )
    .group_by(
        pl.col("decade"),
        maintain_order=True,
    )
    .agg(
        pl.col("name"),
        pl.col("weight", "height").mean().round(2).name.prefix("avg_"),
    )
)

```

```
)
print(result)
```



[API group_by](#) · [API agg](#) · [API select](#) · [API with_columns](#) · [API str namespace](#) · [API list namespace](#) ·
[Available on feature strings](#)

```
let result = df
    .clone()
    .lazy()
    .with_columns([
        (col("birthdate").dt().year() / lit(10) * lit(10)).alias("decade"),
        col("name").str().split(lit(" ")).list().first(),
    ])
    .select([all().exclude_cols(["birthdate"]).as_expr()])
    .group_by([col("decade")])
    .agg([
        col("name"),
        cols(["weight", "height"])
            .as_expr()
            .mean()
            .round(2, RoundMode::default())
            .name()
            .prefix("avg_"),
    ])
    .collect()?;
println!("{result}");
```

shape: (2, 4)

decade	name	avg_weight	avg_height
---	---	---	---
i32	list[str]	f64	f64
1990	["Alice"]	57.9	1.56
1980	["Ben", "Chloe", "Daniel"]	69.73	1.72

Combining dataframes

Polars provides a number of tools to combine two dataframes. In this section, we show an example of a join and an example of a concatenation.

Joining dataframes

Polars provides many different join algorithms. The example below shows how to use a left outer join to combine two dataframes when a column can be used as a unique identifier to establish a correspondence between rows across the dataframes:

Python

API `join`

```
df2 = pl.DataFrame(  
    {  
        "name": ["Ben Brown", "Daniel Donovan", "Alice Archer", "Chloe Cooper"],  
        "parent": [True, False, False, False],  
        "siblings": [1, 2, 3, 4],  
    }  
)  
  
print(df.join(df2, on="name", how="left"))
```

Rust

API `join`

```
let df2: DataFrame = df!(  
    "name" => ["Ben Brown", "Daniel Donovan", "Alice Archer", "Chloe Cooper"],  
    "parent" => [true, false, false, false],  
    "siblings" => [1, 2, 3, 4],  
)  
.unwrap();  
  
let result = df  
    .clone()  
    .lazy()  
    .join(  
        df2.lazy(),  
        [col("name")],  
        [col("name")],  
        JoinArgs::new(JoinType::Left),  
    )  
    .collect()?;  
  
println!("{result}");
```

```
shape: (4, 6)
```

name	birthdate	weight	height	parent	siblings
---	---	---	---	---	---
str	date	f64	f64	bool	i64
Alice Archer	1997-01-10	57.9	1.56	false	3
Ben Brown	1985-02-15	72.5	1.77	true	1
Chloe Cooper	1983-03-22	53.6	1.65	false	4
Daniel Donovan	1981-04-30	83.1	1.75	false	2

Polars provides many different join algorithms that you can learn about in the [joins section of the user guide](#).

Concatenating dataframes

Concatenating dataframes creates a taller or wider dataframe, depending on the method used. Assuming we have a second dataframe with data from other people, we could use vertical concatenation to create a taller dataframe:

Python

[API](#) [concat](#)

```
df3 = pl.DataFrame(
    {
        "name": ["Ethan Edwards", "Fiona Foster", "Grace Gibson", "Henry
Harris"],
        "birthdate": [
            dt.date(1977, 5, 10),
            dt.date(1975, 6, 23),
            dt.date(1973, 7, 22),
            dt.date(1971, 8, 3),
        ],
        "weight": [67.9, 72.5, 57.6, 93.1], # (kg)
        "height": [1.76, 1.6, 1.66, 1.8], # (m)
    }
)

print(pl.concat([df, df3], how="vertical"))
```

Rust

[API](#) [concat](#)

```
let df3: DataFrame = df!(
  "name" => ["Ethan Edwards", "Fiona Foster", "Grace Gibson", "Henry Harris"],
  "birthdate" => [
    NaiveDate::from_ymd_opt(1977, 5, 10).unwrap(),
    NaiveDate::from_ymd_opt(1975, 6, 23).unwrap(),
    NaiveDate::from_ymd_opt(1973, 7, 22).unwrap(),
    NaiveDate::from_ymd_opt(1971, 8, 3).unwrap(),
  ],
  "weight" => [67.9, 72.5, 57.6, 93.1], // (kg)
  "height" => [1.76, 1.6, 1.66, 1.8], // (m)
)
.unwrap();

let result = concat([df.clone().lazy(), df3.lazy()],
  UnionArgs::default()?).collect()?;
println!("{result}");
```

shape: (8, 4)

name	birthdate	weight	height
---	---	---	---
str	date	f64	f64
Alice Archer	1997-01-10	57.9	1.56
Ben Brown	1985-02-15	72.5	1.77
Chloe Cooper	1983-03-22	53.6	1.65
Daniel Donovan	1981-04-30	83.1	1.75
Ethan Edwards	1977-05-10	67.9	1.76
Fiona Foster	1975-06-23	72.5	1.6
Grace Gibson	1973-07-22	57.6	1.66
Henry Harris	1971-08-03	93.1	1.8

Polars provides vertical and horizontal concatenation, as well as diagonal concatenation. You can learn more about these in the [concatenations section of the user guide](#).